

Version Control in AWS Lambda

To Begin with the Lab

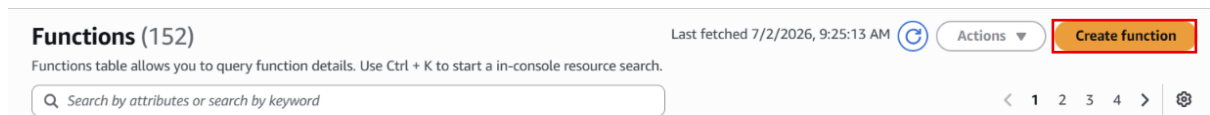
Summary of the Lab

This lab explains AWS Lambda version control as a prerequisite for understanding Provisioned Concurrency. It demonstrates how to publish immutable Lambda versions, test and preserve working code, make updates without losing previous versions, and easily roll back when needed. The video also introduces Lambda aliases, Go the Lambda in your AWS account. How to use Lambda aliases to switch between different published Lambda versions without changing the API Gateway URL. It also demonstrates weighted traffic routing for canary and blue-green deployments, enabling smooth application updates.

Lab steps

Step 1: Create Function

- Create **Lambda Function**
- Click on the **Create Function**

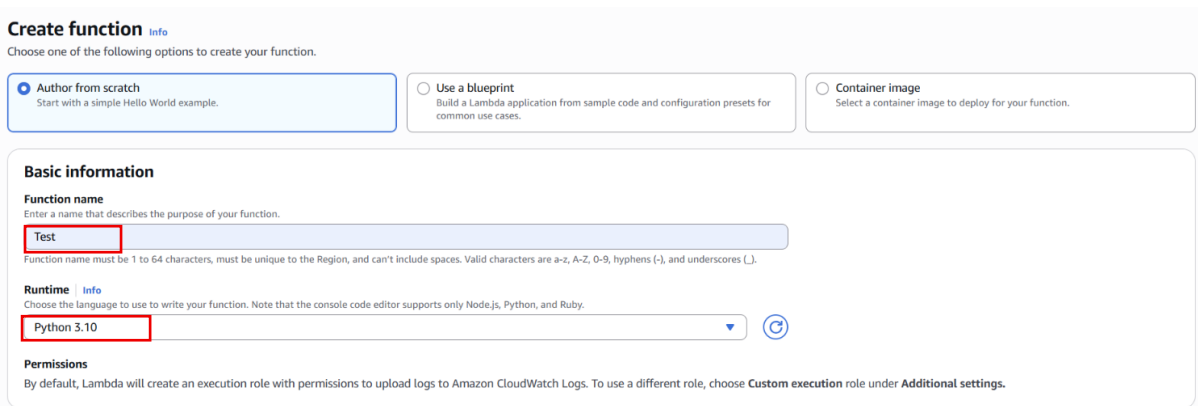


Functions (152) Last fetched 7/2/2026, 9:25:13 AM Actions Create function

Functions table allows you to query function details. Use Ctrl + K to start a in-console resource search.

Search by attributes or search by keyword

- Select **Author from scratch**
- Provide the Function Name **Test** to identify the function
- Select a **python Language**
- Leave all remaining setting **default**



Create function Info

Choose one of the following options to create your function.

☒ Author from scratch Start with a simple Hello World example.

☐ Use a blueprint Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image Select a container image to deploy for your function.

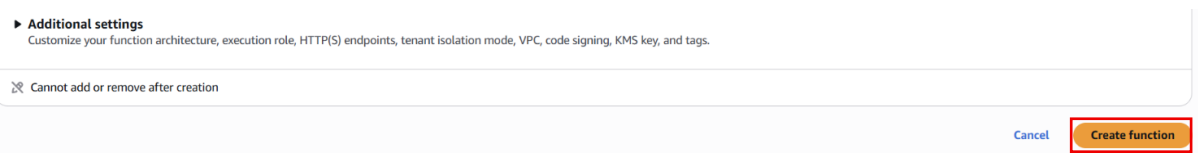
Basic information

Function name
Enter a name that describes the purpose of your function.
Test
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.10

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **Additional settings**.

- Click on the **Create function**



► Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.

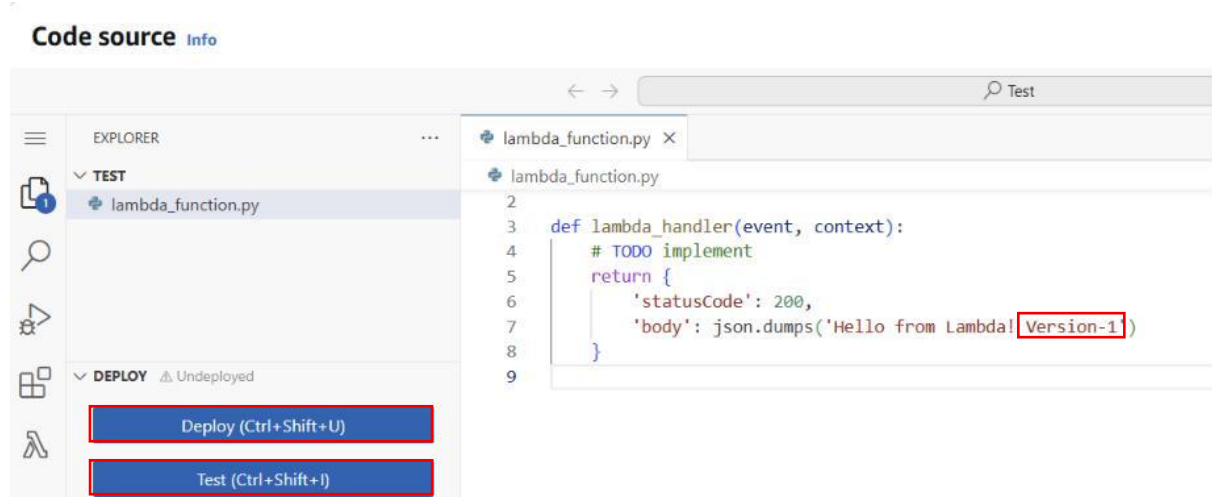
⚠ Cannot add or remove after creation

Cancel Create function

Step 2: Add simple code

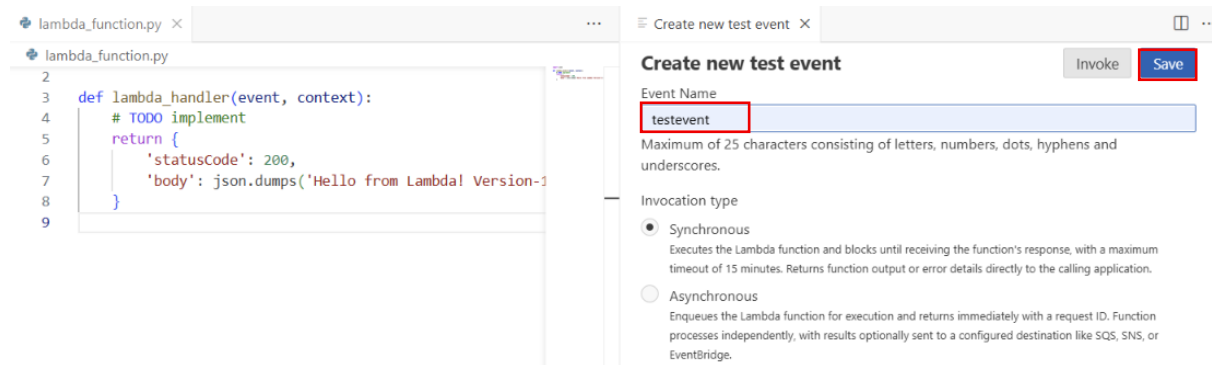
- Go to the code source in your Test Function
- Add **Version-1** in code

- Click on the **Deploy**

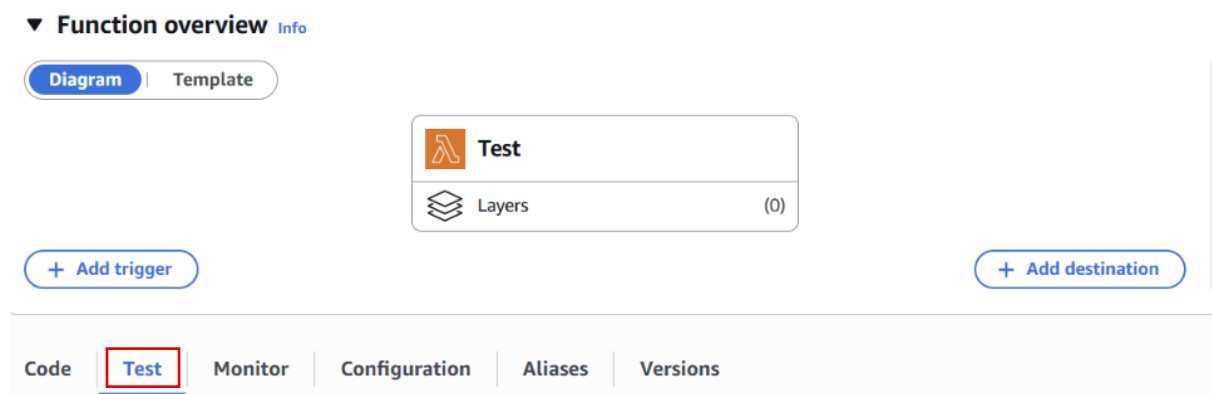


Step 5: Test the Function

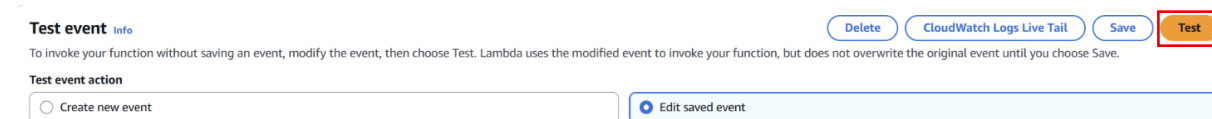
- Click on the **Test** and **Create New Test event**
- Provide Event Name **testevent**
- Save** the Test event



- Go to the **Test** section in Test Function



- Click on the **Test**



- Executing the Test function

- Click on the **details**

Executing function: succeeded ([logs](#))

Details

```
{
  "statusCode": 200,
  "body": "\"Hello from Lambda! version-1\""
}
```

Step 6: Publish Version 1

- Navigate to the **Versions**

Function overview [Info](#)

Diagram | Template

Test

Layers (0)

+ Add trigger + Add destination

Code | **Test** | Monitor | Configuration | Aliases | **Versions**

- Click on the **Publish new version**

Versions (0) [Info](#) Last fetched 7/2/2026, 9:48:33 AM [Delete version](#) [Publish new version](#)

Search by attributes or search by keyword

Version	Aliases	Description	Last modified	Architecture
No versions				
This function does not have any published versions.				

[Publish new version](#)

- Provide version
- Click on the **Publish**

Publish new version from \$LATEST

Publishing a new version saves a snapshot of the code and the configuration of the \$LATEST version. You need to deploy code changes in \$LATEST before you can create a new version. Any triggers you added to the function are not saved to the new version. You can't change the new version's code. Choose Publish to confirm.

Version description - optional

v-1

SnapStart
None

Cancel **Publish**

- Latest Version** in Lambda function
- Incremental number, that highly number is the Latest version

Versions (1) [Info](#)

Last fetched 7/2/2026, 9:50:06 AM

Delete version

Publish new version

Search by attributes or search by keyword

Version

Aliases

Description

Last modified

Architecture

1

-

v-1

6 minutes ago

x86_64

Step 7: Modify the Code

- Change the code
- **Deploy**
- You lose old version so, Create versions
- Test again

[Code](#)
[Test](#)
[Monitor](#)
[Configuration](#)
[Aliases](#)
[Versions](#)

Code source [Info](#)

EXPLORER

TEST

lambda_function.py

Refresh Explorer

DEPLOY

Undeployed

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

lambda_function.py

```

2
3 def lambda_handler(event, context):
4     # TODO implement
5     return {
6         'statusCode': 200,
7         'body': json.dumps('Hello from Lambda! version-2')
8     }
9

```

- Create version v-2

Publish new version from \$LATEST
✕

Publishing a new version saves a snapshot of the code and the configuration of the \$LATEST version. You need to deploy code changes in \$LATEST before you can create a new version. Any triggers you added to the function are not saved to the new version. You can't change the new version's code. Choose Publish to confirm.

Version description - optional

v-2

SnapStart
None

[Cancel](#)
[Publish](#)

- Number of 2 is the **Latest Version**

Versions (2) [Info](#)

Last fetched 7/2/2026, 9:55:31 AM [Refresh](#)

[Delete version](#)

[Publish new version](#)

< 1 >

	Version	Aliases	Description	Last modified	Architecture
☐	2	-	v-2	4 minutes ago	x86_64
☐	1	-	v-1	11 minutes ago	x86_64

- If have you want old version click on version
- Click on the **First Version**

Versions (3) [Info](#) Last fetched 7/2/2026, 10:00:36 AM [Delete version](#) [Publish new version](#)

Search by attributes or search by keyword

	Version	Aliases	Description	Last modified	Architecture
☐	3	-	v-3	1 minute ago	x86_64
☐	2	-	v-2	10 minutes ago	x86_64
☐	1	-	v-1	16 minutes ago	x86_64

- This is the old version
- Click on the **Test**

Function overview [Info](#)

[Diagram](#) [Template](#)

Test:1

Layers (0)

[+ Add trigger](#) [+ Add destination](#)

[Export to Infrastructure Composer](#) [Download](#)

Function ARN
arn:aws:lambda:ap-south-1:463646775279:function:Test:1

Description
v-1

Last modified
18 minutes ago

[Code](#) [Test](#) [Monitor](#) [Configuration](#)

Test event [Info](#) [Delete](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

- Executing the old Version in Lambda Function

Executing function: succeeded ([logs](#))

[Details](#)

```
{
  "statusCode": 200,
  "body": "\"Hello from Lambda! version-1\""
}
```

- Go to the **Aliases**
- Easy deployment: Switch between versions **without changing your application.**

Versions (1) [Info](#) Last fetched 7/2/2026, 9:50:06 AM [Delete version](#) [Publish new version](#)

Search by attributes or search by keyword

	Version	Aliases	Description	Last modified	Architecture
☐	1	-	v-1	6 minutes ago	x86_64

[Code](#) [Test](#) [Monitor](#) [Configuration](#) [Aliases](#) [Versions](#)

Aliases (0) [Info](#) Last fetched 7/2/2026, 11:35:20 AM [Edit](#) [Delete](#) [Create alias](#)

Search by attributes or search by keyword

Name	Versions	Description
No aliases This function does not have any aliases		

[Create alias](#)

- Provide the Aliases Name **pord**
- Choose Version

- Click on the **Save**

Create alias

Alias configuration
An alias is a pointer to one or two versions. Choose each version that you want the alias to point to.

Name
pord

Description - optional

Version
\$LATEST

► Weighted alias

Cancel Save

- Go to the Configuration in Aliases
- Click on the **Trigger**
- Click on the **Add trigger**

Test Monitor **Configuration**

Configuration

General configuration
Triggers
Permissions
Destinations
Function URL
Monitoring and operations tools

Triggers (0) Info

Search for triggers

Trigger

No triggers
No triggers are configured.
Add trigger

Fix errors Edit Delete Add trigger

- Select **API Gateway** in Trigger Configuration
- Click on the **Create a new API**
- Select **HTTP API**
- Choose **open** in security
- Click on the **Add**

Add trigger

Trigger configuration info

API Gateway
aws api application-services backend HTTP REST serverless

Add an API to your Lambda function to create an HTTP endpoint that invokes your function. API Gateway supports REST, HTTP, and WebSocket APIs. [Learn more](#)

Intent
Use an existing api or have us create one for you.
☒ Create a new API
☐ Use existing API

API type
☒ HTTP API
Build a REST API and cost-effective REST APIs with built-in features such as OIDC and OAuth2, and native CORS support.
☐ WebSocket API
Build a WebSocket API using persistent connections for real-time use cases such as chat applications or dashboards.
☐ REST API
Develop a REST API where you gain complete control over the request and response along with API management capabilities.

Security
Configure the security mechanism for your API endpoint.
Open

► Additional settings

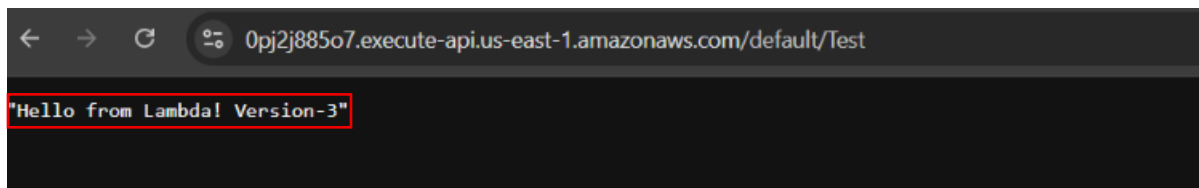
Lambda will add the necessary permissions for Amazon API Gateway to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

Cancel Add

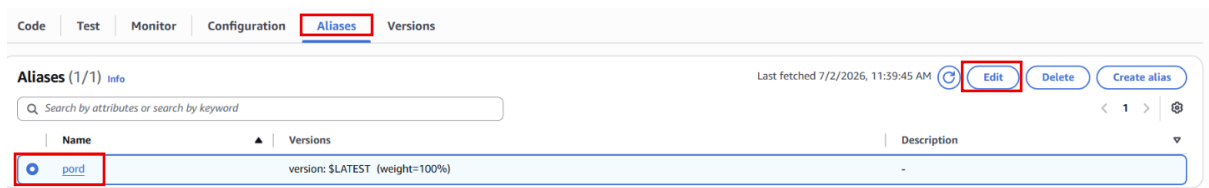
- Copy API Gateway endpoint



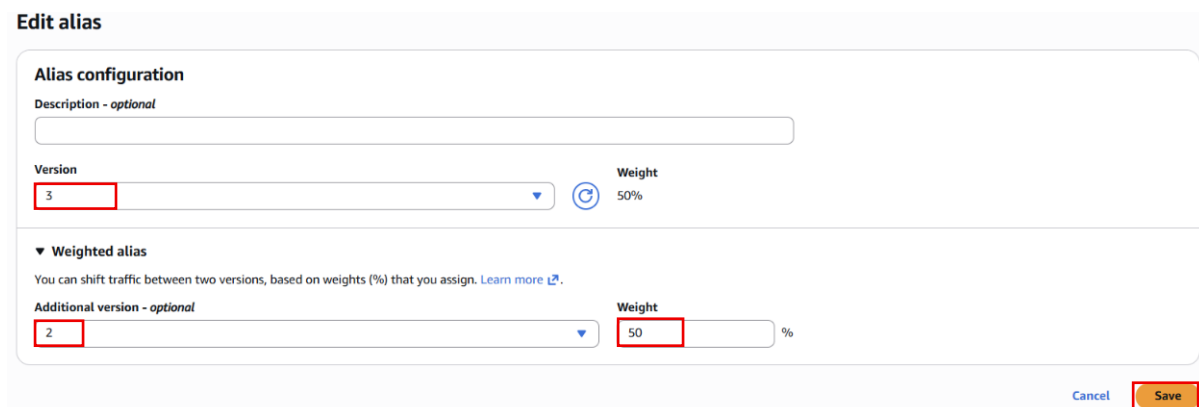
- Paste in your browser
- Execute the latest version of the lambda function in browser with help of API Gateway



- Go to the **Aliases**
- Select **pord** Aliases
- Click on the **edit**



- Select Version
- Click on the **Weighted alias**
- Select Additional version, that you are execute of the version
- Select **weight 50%**
- Click on the **Save**



- Copy the API Gateway **endpoint**



- Paste in your browser
- Refresh your page
- Aliases is Create traffic
- Who you have select Execute the Another Version

